

第五次实验

一、实验目的

1. 更正 memtest.vhd 的代码设计, 使得 rdmem 和 wrdy 信号不会丢失, 并能判断内存读写状态的空满。
2. 学习 sramtop 工程的代码, 并下载到开发板上, 在电脑上验证简要功能。

二、实验内容和步骤

1. 修改 memtest.vhd 代码设计, 使得代码能完成以下目标:
 - 每一个 outen 信号产生一个 rdmem 信号。
 - wrdy 信号不会因为状态机变化的延迟而丢失。
 - 代码可以判断内存是否空满, 当内存为空时 outen 输出信号无效。
2. 将 sram 工程下载到开发板上, 并在电脑上验证开发板的基础功能。

三、实验结果

1. 添加类似 wrdy 信号的读信号 rdrdy 来生成 rdmem 信号, 防止 rdmem 信号丢失。

类似于输入计数后产生 wrdy 信号, 定义一个标志位 rdrdy 来生成读信号,

定义一个两位寄存器 outensft 来保持长周期 outen 读取需求。

```
signal rdrdy :std_logic; -- 用来产生读取信号
```

```
signal outensft:std_logic_vector(1 downto 0); -- 用于储存长周期 outen 信号
```

```

process(rst,clk)

begin

    if rst = '0' then

        outensft <= "00";

    elsif clk'event and clk = '1' then

        outensft(1) <= outensft(0);

        outensft(0) <= outen;

    end if;

end process;

```

```

process(rst,clk)

begin

    if rst = '0' then

        rdrdy <= '0';

    elsif clk'event and clk = '1' then

        if outensft = 1 then

            rdrdy <= '1';

        elsif ss = s2 then

            rdrdy <= '0';

        end if;

    end if;

end process;

```

再修改状态机 ss 的代码，将 ss 进入 s2 读状态的条件改写为：

```
case ss is

    when s0 =>    -- 空闲状态

        if wrdy = '1' then    -- 写数据准备好，转移到写状态

            ss <= s1;

        elsif rdy = '1' then -- 输出请求有效，转移到读状态

            ss <= s2;

        end if;

    when s1 =>    -- 写内存状态，完成后返回空闲状态（s1,s2 只存在
一个时钟节拍）

        ss <= s0;

    when s2 =>    -- 读内存状态，完成后返回空闲状态

        ss <= s0;

    when others => -- 异常状态处理，返回空闲状态

        ss <= s0;

end case;
```

2. 修改状态机中 wrdy 信号的清零逻辑，防止 wrdy 丢失。

```
process(rst,clk)

begin

    if rst = '0' then

        wrdy <= '0'; -- 复位时写准备好信号无效

    elsif clk'event and clk = '1' then
```

```

        if rxdcnt = 8 then    -- 已接收 8 位数据，设置准备好信号

            wrdy <= '1';

        elsif ss = s1 then    -- 处于写状态时清除准备好信号

            wrdy <= '0';

        end if;

    end if;

end process;

```

3. 增加判断内存空满状态的代码逻辑。

当 rdmemaddr = wrmemaddr 时，读写地址存在两个状态：

内存地址为满 或 内存地址为空。

因为当内存地址为满时，任何写操作都会导致机器出错，将内存满状态视为不可取的错误状态，outen 无效。同时我们要求内存为空状态时 outen 也无效，则只需判断 rdmemaddr = wrmemaddr 时，outen 无效即可。

在状态机 ss 进入 s2 的条件中附加上 rdmemaddr /= wrmemaddr 即可。

```

case ss is

    when s0 =>    -- 空闲状态

        if wrdy = '1' then    -- 写数据准备好，转移到写状态

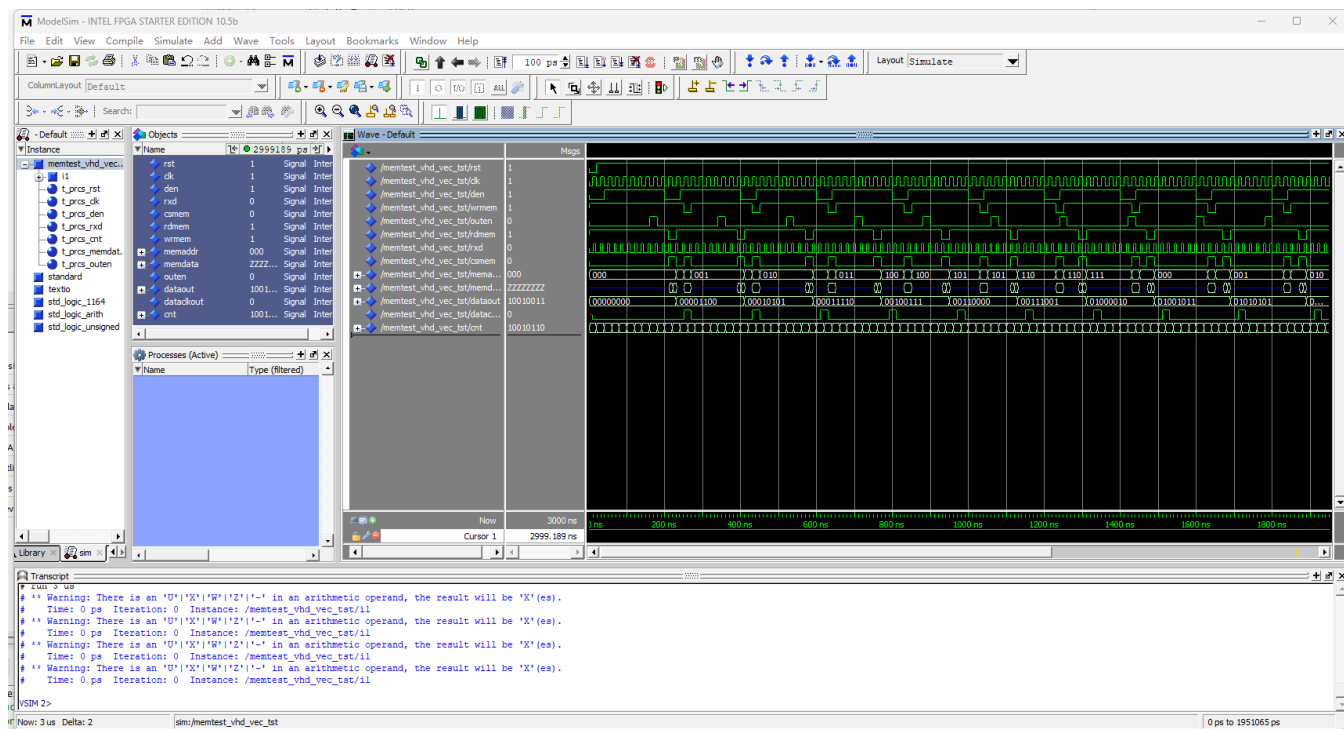
            ss <= s1;

        elsif rdy = '1' and rdmemaddr /= wrmemaddr then -- 输出请求有效，转移到读状态

            ss <= s2;

```

when s1 => -- 写内存状态，完成后返回空闲状态 (s1,s2 只存在一个时钟节拍)



四、思考题

1. 可综合代码和不可综合代码的区别和应用场景是什么？

Verilog HDL 或 VHDL 都是一种硬件的描述语言，并且允许在不同的抽象层

次上对电路进行建模。综合就是将语言描述的 RTL 级的电路模型构造出门级网表的过程。综合后生成的网表文件，就是由导线相互连接的寄存器传输级功能块（比如触发器、算术逻辑单元和多路选择器等）组成的。

HDL 的描述有不同的抽象层次，从门级、RTL 级、行为级以及算法级进行建模。但是，并不是所有层次上的描述方式都可以被综合成想要的硬件元件，即不可综合。比如 Verilog 中存在一些用于仿真验证的子集，属于仿真验证语言，只在仿真时候使用，不能被综合成电路，因为没有相应的硬件元件与其对应。如 initial 语句。

2. 阻塞赋值和非阻塞赋值的电路结构区别是什么？

阻塞赋值"="，必须是阻塞赋值完成后，才进行下一条语句的执行；赋值一旦完成，等号左边的变量值立即变化。在同一个块中，阻塞赋值表达式的书写顺序会影响赋值的结果。硬件没有对应的电路。

非阻塞赋值"<="，在赋值开始时计算表达式右边的值，在本次仿真周期时钟的下降沿时才更新被赋值变量，即赋值不是立即生效的；非阻塞赋值允许块中其他语句同时执行。在同一个块中，非阻塞赋值表达式的书写顺序不影响赋值的结果。硬件有对应的电路。

阻塞赋值对应的电路结构往往与触发沿没有关系，只与输入的电平变化有关系；非阻塞赋值对应的电路结构往往与触发沿有关系，只有在触发沿时才有可能发生赋值的情况。

3. 写出交通灯例子的测试激励代码。

```
LIBRARY ieee;  
  
USE ieee.std_logic_1164.all;
```

```
ENTITY tlight28_sim_vhd_vec_tst IS

END tlight28_sim_vhd_vec_tst;

ARCHITECTURE tlight28_sim_arch OF tlight28_sim_vhd_vec_tst IS

-- constants

-- signals

SIGNAL CLK : STD_LOGIC;

SIGNAL entm3 : STD_LOGIC;

SIGNAL entm40 : STD_LOGIC;

SIGNAL entm60 : STD_LOGIC;

SIGNAL ET : STD_LOGIC;

SIGNAL EWGREEN : STD_LOGIC;

SIGNAL EWRED : STD_LOGIC;

SIGNAL EYWELLOW : STD_LOGIC;

SIGNAL NSGREEN : STD_LOGIC;

SIGNAL NSRED : STD_LOGIC;

SIGNAL NSYELLOW : STD_LOGIC;

SIGNAL NT : STD_LOGIC;

SIGNAL RST : STD_LOGIC;

SIGNAL S : STD_LOGIC_VECTOR(1 DOWNTO 0);

SIGNAL ST : STD_LOGIC;

SIGNAL timer_3 : STD_LOGIC_VECTOR(1 DOWNTO 0);
```

SIGNAL timer_40 : STD_LOGIC_VECTOR(5 DOWNT0 0);

SIGNAL timer_60 : STD_LOGIC_VECTOR(5 DOWNT0 0);

SIGNAL tm3 : STD_LOGIC;

SIGNAL tm40 : STD_LOGIC;

SIGNAL tm60 : STD_LOGIC;

SIGNAL WT : STD_LOGIC;

COMPONENT tlight28_sim

PORT (

CLK : IN STD_LOGIC;

entm3 : OUT STD_LOGIC;

entm40 : OUT STD_LOGIC;

entm60 : OUT STD_LOGIC;

ET : IN STD_LOGIC;

EWGREEN : OUT STD_LOGIC;

EWRED : OUT STD_LOGIC;

EWYELLOW : OUT STD_LOGIC;

NSGREEN : OUT STD_LOGIC;

NSRED : OUT STD_LOGIC;

NSYELLOW : OUT STD_LOGIC;

NT : IN STD_LOGIC;

RST : IN STD_LOGIC;

S : OUT STD_LOGIC_VECTOR(1 DOWNT0 0);


```

    ST : IN STD_LOGIC;

    timer_3 : OUT STD_LOGIC_VECTOR(1 DOWNTO 0);

    timer_40 : OUT STD_LOGIC_VECTOR(5 DOWNTO 0);

    timer_60 : OUT STD_LOGIC_VECTOR(5 DOWNTO 0);

    tm3 : OUT STD_LOGIC;

    tm40 : OUT STD_LOGIC;

    tm60 : OUT STD_LOGIC;

    WT : IN STD_LOGIC

);

END COMPONENT;

BEGIN

    i1 : tlight28_sim

    PORT MAP (

-- list connections between master ports and signals

        CLK => CLK,

        entm3 => entm3,

        entm40 => entm40,

        entm60 => entm60,

        ET => ET,

        EWGREEN => EWGREEN,

        EWRED => EWRED,

        EYWELLOW => EYWELLOW,

```

```
NSGREEN => NSGREEN,  
  
NSRED => NSRED,  
  
NSYELLOW => NSYELLOW,  
  
NT => NT,  
  
RST => RST,  
  
S => S,  
  
ST => ST,  
  
timer_3 => timer_3,  
  
timer_40 => timer_40,  
  
timer_60 => timer_60,  
  
tm3 => tm3,  
  
tm40 => tm40,  
  
tm60 => tm60,  
  
WT => WT  
  
);
```

```
-- RST
```

```
t_prcs_RST: PROCESS
```

```
BEGIN
```

```
    RST <= '1';
```

```
WAIT;
```

```
END PROCESS t_prcs_RST;
```

-- CLK

t_prcs_CLK: PROCESS

BEGIN

LOOP

CLK <= '0';

WAIT FOR 5000 ps;

CLK <= '1';

WAIT FOR 5000 ps;

IF (NOW >= 1000000 ps) THEN WAIT; END IF;

END LOOP;

END PROCESS t_prcs_CLK;

-- ET

t_prcs_ET: PROCESS

BEGIN

ET <= '1';

WAIT;

END PROCESS t_prcs_ET;

-- WT

t_prcs_WT: PROCESS

```

BEGIN

    WT <= '0';

WAIT;

END PROCESS t_prcs_WT;


-- ST

t_prcs_ST: PROCESS

BEGIN

    ST <= '0';

WAIT;

END PROCESS t_prcs_ST;


-- NT

t_prcs_NT: PROCESS

BEGIN

    NT <= '1';

WAIT;

END PROCESS t_prcs_NT;

END tlight28_sim_arch;

```

4. 画出 DE2-115 上面能跑的 lab2/step2/sram 工程的电路结构框图，尽量详细。

